# 计算机学院实验报告

| 实验题目： 三种平面三角形点定位算法 | | 学号：202200130077 |
|---|---|---|
| 日期：2024. 3. 23 | 班级：智能 | 姓名：薛润泽 |
| **Email：** | | |

实验目的：

• [5 分] 正确构建模型矩阵。
• [5 分] 正确构建透视投影矩阵。
• [10 分] 你的代码可以在现有框架下正确运行，并能看到变换后的三角形。
• [10 分] 当按 A 键与 D 键时，三角形能正确旋转。或者正确使用命令行得 到旋转结果图像。
• [提高项 5 分] 在 main.cpp 中构造一个函数，该函数的作用是得到绕任意过原点的轴的旋转变换矩阵。
Eigen::Matrix4f get_rotation(Vector3f axis, float angle)

实验环境介绍：

Ubuntu22.04

**解决问题的主要思路：**

1、对于绕 z 轴旋转，旋转矩阵为:

$$\mathcal{R}_z(\theta_z) = \begin{bmatrix} \cos\theta_z & -\sin\theta_z & 0 \\ \sin\theta_z & \cos\theta_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2、对于绕任意过原点的轴，我们有罗德里格斯公式

$R = I + \sin(\theta)K + (1 - \cos(\theta))K$

其中，I 是单位矩阵，K 是一个由我们所绕的旋转轴定义的斜对称矩阵， θ 是我们要旋转的角度。K 斜对称矩阵如下，其中 kx，ky，kz 分别对应过原点的旋转轴向量的 x，y，z。

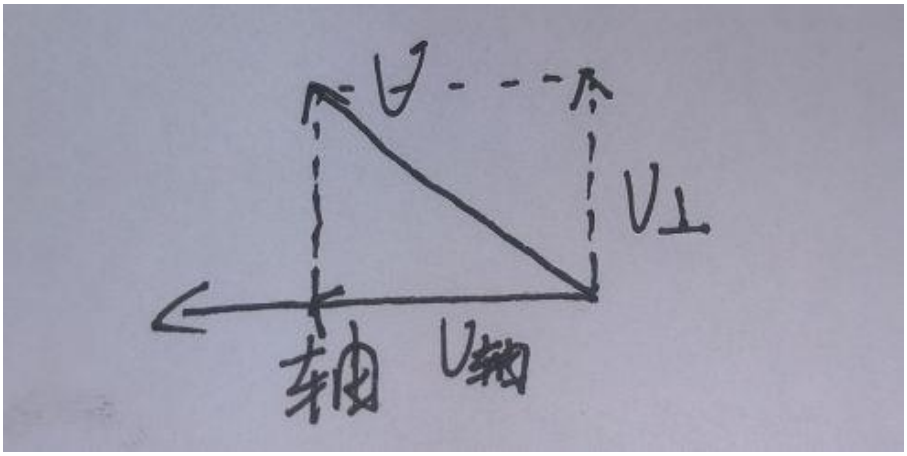
$$K = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

3、本实验的主要思路如下：

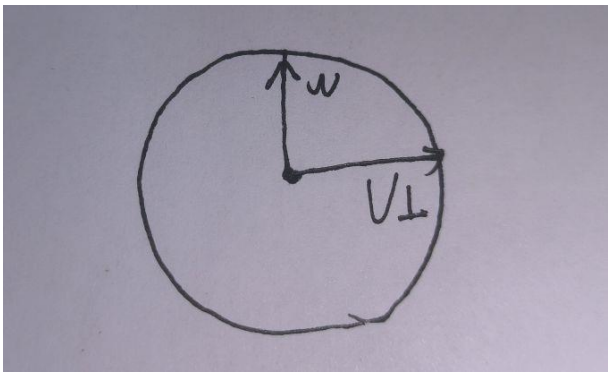
1、使用绕 Z 轴旋转的旋转矩阵。使用 c++完成旋转矩阵的构建。
2、使用罗德里格斯公式实现过原点的任意轴旋转，并得出旋转矩阵。
3、实现透视投影的投影矩阵。

**罗德里格斯公式推导思路：**其推导的主要思路是旋转一个向量可以看做一个向量的分解向量旋转，然后将旋转完成后的分解向量重新合成。

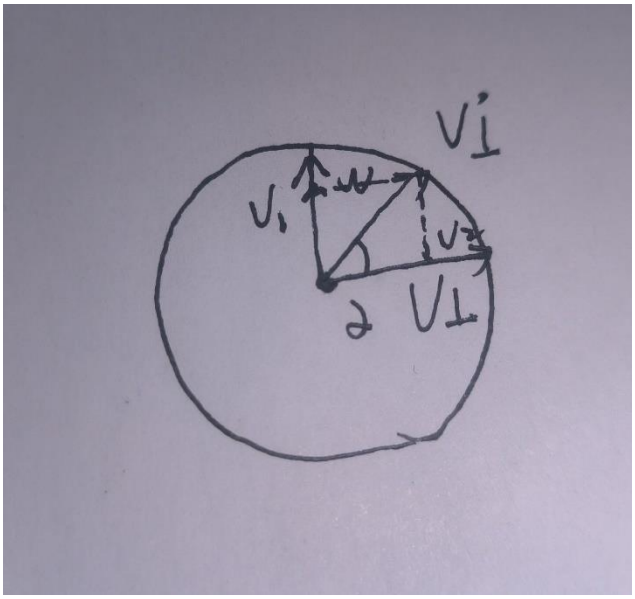
罗德里格斯公式选择将原向量分解到旋转轴和与旋转轴垂直的方向上。分解到旋转轴上的向量在旋转过程中保持不变，所以只需要考虑分解到与旋转轴垂直方向上的向量（v⊥）的旋转就好。分解如下图（轴向量未表示出，v 轴是 v 向量在旋转轴上的分解向量，v⊥是 v 向量垂直于旋转轴的分解向量）：




使用旋转轴方向上的一个单位向量 k，计算 w = k×v⊥，得到如下图：




旋转轴垂直与上圆圆心，垂直纸面向外。因为 k 是单位向量，所以 w 的模长与 V⊥模长相等。所以，V⊥旋转即绕过圆心的轴进行旋转，示意图如下：




**所以，V'⊥就是 V⊥的旋转结果向量。**

$$V_{\perp}' = \cos(\alpha)\, V_{\perp} + \sin(\alpha)\mathrm{w}$$

**其中**

$$w = k \times V_{\perp}$$

$$V_{轴} = (\mathrm{k} \cdot \mathrm{V}) \cdot \mathrm{k}$$

$$V_{\perp} = V - V_{轴} = V - (k \cdot V) \cdot k$$

$$V' = V_{轴} + V_{\perp}' = (\mathrm{k} \cdot \mathrm{V}) \cdot \mathrm{k} + \cos(\alpha)\,(V - (k \cdot V) \cdot k) + \sin(\alpha)(k \times V - (k \cdot V) \cdot k)$$

**即**

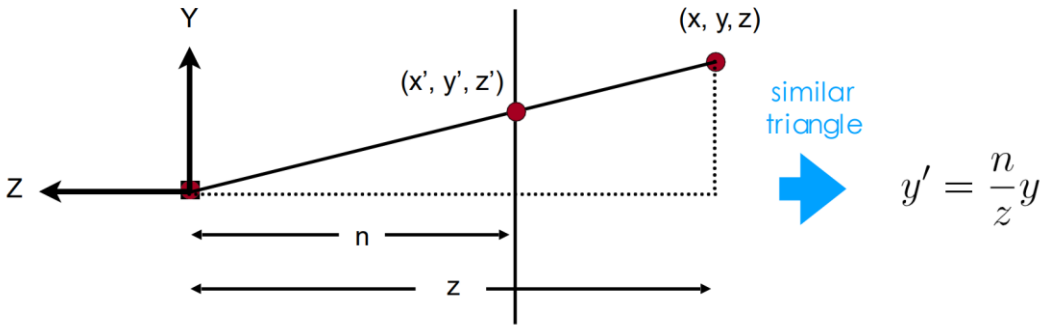
$$V' = \cos(\alpha)\, V + (1 - \cos(\alpha))(V \cdot k) \cdot k + \sin(\alpha)\, k \times V$$

上述 V' 式子就是绕任意过原点轴的旋转公式，化作矩阵形式为：

$$R = \cos\theta I + (1 - \cos\theta) K K^{T} + \sin\theta K^{\wedge}$$

**投影矩阵实现原理**

如下图：




根据三角形相似原理，可以得到

$$y' = \frac{n}{z}y$$

$$z' = \frac{n}{z}z$$

想要得到 z 的信息，需要取两个特殊点进行运算。第一个特殊点是(x, y, n,1)；

第二个特殊点(0,0,f,1)；我们通过对第一个点的带入，发现最后的旋转矩阵第三

行一定是(0,0,A,B)且有

$$An + B = n^2$$

带入第二个点有

$$Af + B = f^2$$

通过解二元方程从而解得 A，B 值

实验步骤与实验结果：

**1. 绕 z 轴旋转的代码实现**

```
Eigen::Matrix4f get_model_matrix(float rotation_angle)
{
    Eigen::Matrix4f model = Eigen::Matrix4f::Identity();
    model<<cos(rotation_angle),-sin(rotation_angle),0, 0,
     sin(rotation_angle),cos(rotation_angle),0,0,
     0,0,1.0f,0,
     0,0,0,1.0f;
    return model;
}
```

**2. 绕任意轴旋转的代码实现**

```
Eigen::Matrix4f get_rotation(Vector3f axis, float angle){
    Eigen::Matrix3f model;
    double uinit = sqrt(axis[0]*axis[0] + axis[1]*axis[1] +
axis[2]*axis[2]);
    axis = axis/uinit;
    model<<
    0, -axis[2], axis[1],
    axis[2], 0, -axis[0],
    -axis[1], axis[0], 0;
    Eigen::Matrix3f I = Eigen::Matrix3f::Identity();
    Eigen::Matrix3f result = I + sin(angle)*model + (1-
cos(angle))*model*model;
    Eigen::Matrix4f final2;
    final2<<
    result(0,0), result(0,1), result(0,2), 0,
    result(1,0), result(1,1), result(1,2), 0,
    result(2,0), result(2,1), result(2,2), 0,
    0, 0, 0, 1;
    return final2;
}
```

### 3. 投影矩阵实现

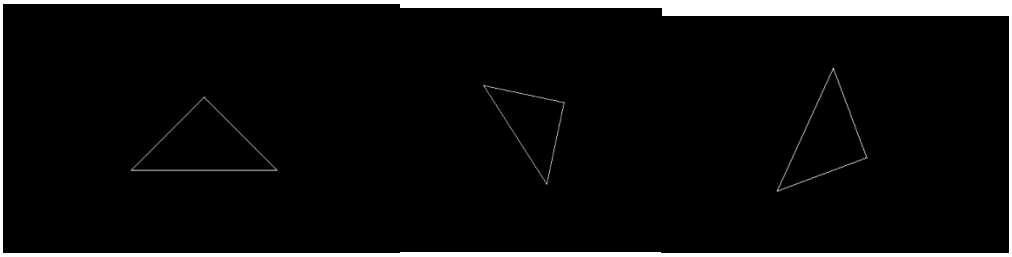
```
Eigen::Matrix4f get_projection_matrix(float eye_fov, float
aspect_ratio,
                                      float zNear, float zFar)
{
    // Students will implement this function
    float n = zNear;
    float f = zFar;
    float t = zNear*tan(eye_fov/2);
    float b = -zNear*tan(eye_fov/2);
    float r = aspect_ratio*zNear*tan(eye_fov/2);
    float l = -aspect_ratio*zNear*tan(eye_fov/2);
    Eigen::Matrix4f projection = Eigen::Matrix4f::Identity();

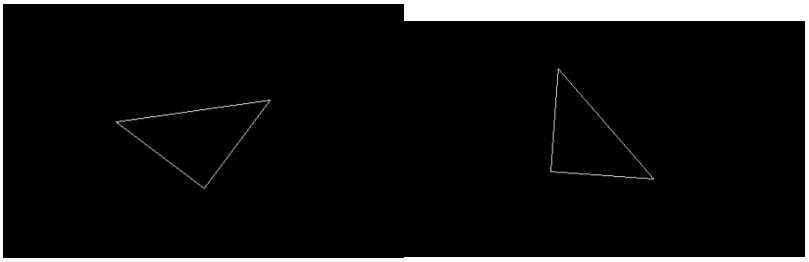
    projection<<
    2*n/(r-l), 0, (r + l)/(r - l), 0,
    0, 2*n/(t-b), (t+b)/(t-b), 0,
    0, 0, -(f + n)/(f - n), -2*f*n/(f - n),
    0, 0, -1, 0;

    return projection;
}
```

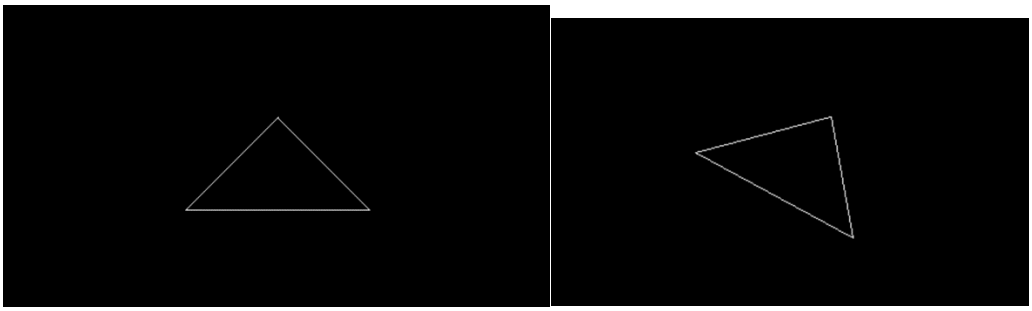
4、实验结果：

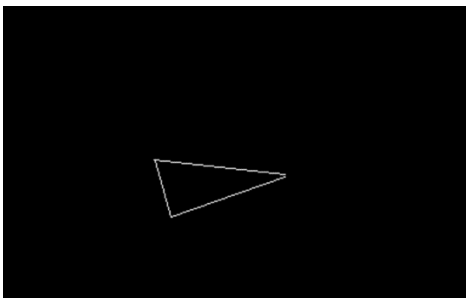
（1）绕 z 轴旋转

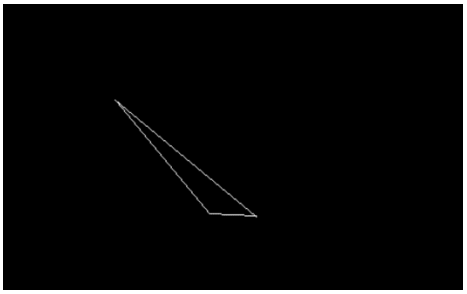


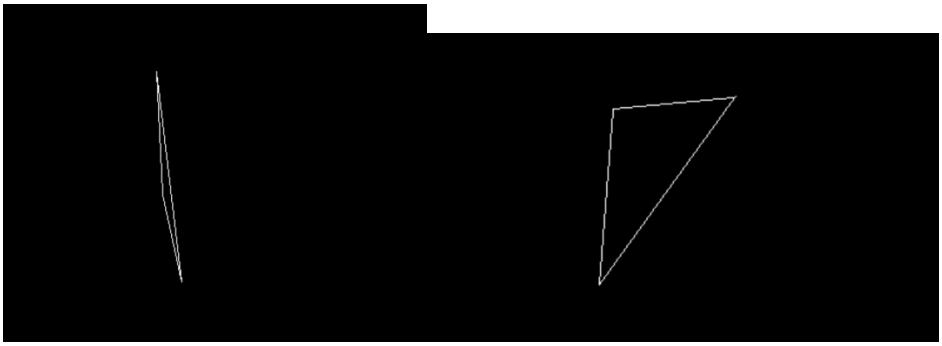


（2）绕轴(1,1,1)旋转（任意轴举例）









**实验中存在的问题及解决：**

问题 1：在绕任意轴旋转的旋转矩阵实现中，旋转轴向量要不要是单位向量。

答 1：在罗德里格斯公式实现旋转矩阵下，旋转轴向量要求是单位向量，如果不是单位向量，在程序运行过程中会发生段错误。猜测过程中可能出现类似除 0 等异常错误。

问题 2：相机观察视角会对程序运行产生影响吗？

答 2：会，在程序运行前，必须要把观察视角拉高，不然会无法观察到模型全部的投影信息，在程序运行过程中出现段错误。